

CS747 – Programming Assignment 3

Avinash Chaudhari (23B1064)

November 2025

1 Algorithm

For all tasks, I implemented the **Minimax algorithm** to determine the best move for a given board position. The algorithm explores the game tree up to a specified search depth (**DEPTH**), evaluates the board positions at the leaf nodes using an evaluation function, and then propagates the resulting scores upward.

At each level:

- The **maximizing player** (White) chooses the move with the maximum evaluation score.
- The **minimizing player** (Black) chooses the move with the minimum evaluation score.

This recursive process continues until the algorithm reaches the maximum search depth or a terminal game state.

2 Evaluation Function

The evaluation function estimates the utility of a given board configuration from White's perspective. Each piece type is assigned a fixed weight based on its relative strength. The material balance is then computed as:

$$\begin{aligned}\text{Material Score} = & w_P \times (W_P - B_P) + w_N \times (W_N - B_N) \\ & + w_B \times (W_B - B_B) + w_R \times (W_R - B_R) \\ & + w_Q \times (W_Q - B_Q) + w_K \times (W_K - B_K)\end{aligned}$$

where W_X and B_X denote the number of White and Black pieces of type X , respectively.

Piece	Weight
Pawn	1
Knight	3
Bishop	3
Rook	5
Queen	9
King	1000

Although the King is never captured, it is included with a high weight to emphasize its importance in evaluation. Initially, I experimented with adding a **mobility term** (based on the number of legal moves available), which improved positional understanding but significantly slowed down computation due to repeated move generation. Therefore, I excluded the mobility term from the final implementation.

3 Optimization: Alpha Beta Pruning

To improve performance, I extended the Minimax algorithm with **Alpha Beta Pruning**. This technique prunes branches that cannot influence the final decision, thereby reducing the number of nodes explored without affecting correctness.

- α : Best value that the maximizing player (White) can guarantee.
- β : Best value that the minimizing player (Black) can guarantee.

During search, if $\alpha > \beta$, further exploration along that branch is terminated (pruned). This significantly reduces the computation time, especially at higher depths.

4 Tasks and Results

The same Minimax algorithm with Alpha Beta Pruning was used across all tasks, differing only in the search depth:

Task	Depth (DEPTH)
Task 1	2
Task 2	3
Task 3	4

All tasks employed the same evaluation function.

In preliminary experiments:

- With the mobility term included in Task 1, the agent achieved approximately **80 wins**, but move generation was noticeably slower.
- Without the mobility term, the agent achieved around **67 wins**, but played much faster.
- For Tasks 2 and 3, removing the mobility term improved both the win rate and response time.

Overall, the final configuration (material based evaluation with Alpha–Beta Pruning) provided a good balance between performance and accuracy.

References

- Chess Programming Wiki — Reference for evaluation heuristics and algorithmic design.